

Контекстуальное встраивание за пределами золотого отрывка

Новый метод обучения, модель и эталон для поиска ответов и контекста, в котором они даны.

30 СЕНТЯБРЯ 2026 Г.

АВТОРЫ Perplexity Research и turbopuffer

Введение

Контекстуальные эмбединги отражают смысл каждого отрывка или фрагмента в контексте всего документа. При их обучении и оценке обычно предполагается наличие одного релевантного фрагмента на запрос, так называемого «золотого фрагмента». Однако на практике золотого фрагмента часто недостаточно. Он может содержать ответ, но в нем может отсутствовать контекст, необходимый для его понимания или проверки, что делает его неоднозначным в отрыве от контекста.

Мы представляем `pp1x-embed-v2-context-9b-preview` нашу новую модель контекстного эмбединга. Модель обучается с использованием нового подхода, в котором модель сжатия контекста от Perplexity выступает в роли учителя. Мы объединяем ее прогнозы на уровне токенов в оценки релевантности на уровне фрагментов, обучая модель эмбединга извлекать как фрагменты ответа, так и вспомогательный контекст, а не один эталонный фрагмент. Модель генерирует по одному эмбеддингу для каждого фрагмента без дополнительных затрат на

логический вывод и поддерживает 1024-мерные эмбединги и эмбединги типа int8.

Она показывает лучшие результаты на **context-bench**, новом бенчмарке для контекстно-зависимого поиска, созданном и находящемся в частной собственности turbopuffer, а также на ConTEB, широко используемом общедоступном бенчмарке. Наша модель была разработана независимо от context-bench и прошла оценку вслепую.

Context-bench состоит из 2099 запросов к 38 894 длинным документам в 21 предметной области и оценивает три возможности контекстных эмбедингов: устранение неоднозначности документов, близких к дубликатам, извлечение «золотых» фрагментов, смысл которых зависит от отдаленного контекста, и воспроизведение подтверждающих доказательств, необходимых для подтверждения ответа.

Предварительный просмотр модели доступен в открытом доступе на Hugging Face. Эталонный тест находится в частном владении компании turbopuffer, чтобы снизить риск загрязнения обучающих данных и сохранить его ценность для оценки прогресса. Чтобы запросить оценку, пожалуйста, свяжитесь с turbopuffer по адресу contextbench@turbopuffer.com.

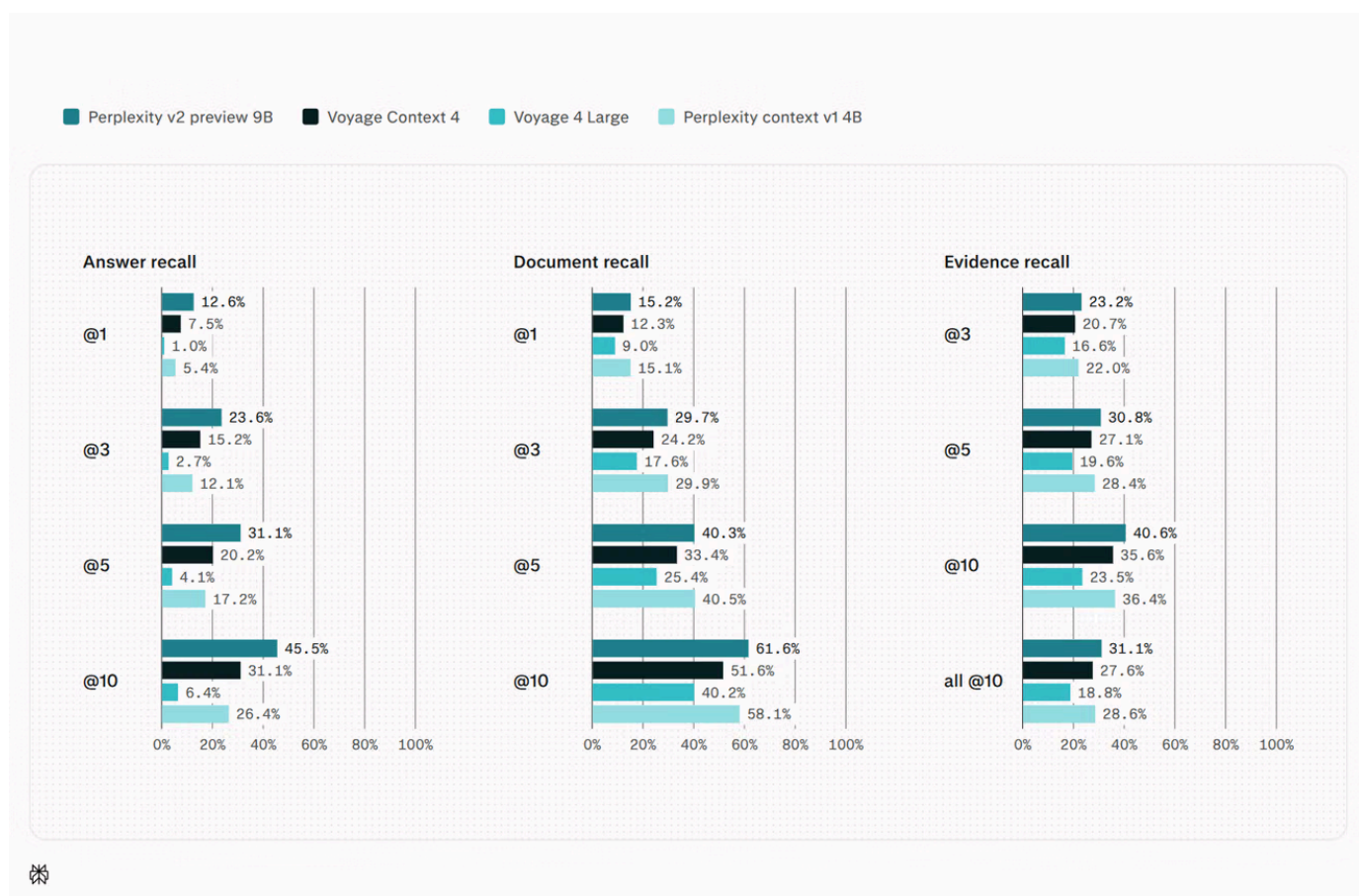


Рис. 1. Answer@K, Document@K, Evidence Recall@K и All-Evidence@10 на context-bench. Сравнение предварительной версии Perplexity v2 9B с Voyage Context 4, Voyage 4 Large и Perplexity context v1 4B.

Почему контекст важен для поиска

Поисковые системы обычно разбивают длинные документы на более мелкие фрагменты, которые можно индексировать и искать по отдельности. Фрагментация удобна на практике, но она лишает нас контекста, в котором изначально находился каждый фрагмент, создавая компромисс между степенью сжатия информации и контекстом, используемым при кодировании. Фрагмент может ссылаться на объект, упомянутый ранее, наследовать свое значение от заголовка раздела или опираться на определения, расположенные в других частях документа. После извлечения фрагмент может не содержать достаточно информации для надежного сопоставления с запросом. Кодирование всего документа в виде одного вектора не решает эту проблему, поскольку одно объединенное представление не может точно отразить все аспекты длинного документа.

Модели контекстного встраивания устраняют это ограничение, представляя каждый фрагмент в контексте окружающего его контента, чаще всего с

помощью позднего разбиения на фрагменты: документ кодируется за один проход, а представления фрагментов объединяются, так что вектор каждого фрагмента вычисляется с учетом всего документа. Предыдущие работы, такие как ConTEB и наша собственная pplx-embed-context-v1, показали, что контекстуализированные представления фрагментов значительно улучшают поиск по сравнению с независимо закодированными фрагментами, особенно в длинных документах.

Ограничения метода «золотых фрагментов»

Для обучения моделей контекстного встраивания требуется контроль, позволяющий определить, какие части документа релевантны данному запросу. Распространенный подход — аннотирование «золотых фрагментов»: для каждого запроса аннотатор, обычно большая языковая модель (БЯМ), определяет фрагмент документа, который является ответом на запрос.

При обучении обычно используются две контрастные функции потерь: **междокументная функция потерь**, которая противопоставляет «золотой» фрагмент фрагментам из других документов, и **внутридокументная функция потерь**, которая противопоставляет его фрагментам из того же документа.

У этого подхода есть несколько ограничений:

- 1. Релевантен только один фрагмент.** Все остальные фрагменты документа, в том числе содержащие полезную вспомогательную информацию, считаются негативными. Контроль на уровне документа — это другая крайность. Документ считается релевантным без указания того, какие фрагменты в нем важны. На практике релевантность часто находится где-то посередине. Фрагмент, содержащий ответ, может зависеть от нескольких вспомогательных фрагментов, которые представляют объект, раскрывают ссылку или содержат дополнительный факт, в то время как остальная часть документа не имеет отношения к делу. Таким образом, модель должна научиться определять правильный уровень избирательности: он должен быть шире, чем у одного «золотого» фрагмента, но уже, чем у документа в целом.

2. **Грубый контроль.** Поскольку каждая аннотация указывает на один «золотой» фрагмент, а не на степень релевантности по всему документу, она дает ограниченное количество обучающих сигналов, учитывая объем вычислительных ресурсов, затраченных на ее создание.
3. **Дорого в масштабировании.** Для каждой обучающей пары требуется, чтобы языковая модель прочитала документ и выделила соответствующий фрагмент, поэтому стоимость аннотирования линейно возрастает с увеличением размера обучающей выборки и ограничивает объем и разнообразие обучающих данных.
4. **Зависит от границ фрагментов.** The labels are also tied to the chunk boundaries used during annotation. A gold label assigned to a fixed token-size window does not specify which shorter sentences or longer paragraphs are relevant. Changing the chunking strategy, or training across multiple chunkings to improve robustness to how documents are split at indexing time, can therefore require annotating the corpus again.

A novel approach to contextual learning

We address the limitations of gold-chunk supervision by distilling relevance judgments from a query-aware context compression model. This model serves as a teacher: it reads the query and document jointly and assigns each document token a relevance score. Its predictions offer several advantages for training contextual embeddings:

1. **Flexible chunk boundaries.** Token-level relevance scores can be aggregated over any chosen chunk boundaries, allowing the same teacher predictions to supervise different chunking without re-annotation.
2. **Continuous relevance.** Unlike binary gold-chunk labels, the teacher provides continuous scores that capture degrees of relevance that naturally distinguish answer-bearing chunks, supporting context, and irrelevant content.
3. **Lower annotation cost.** The teacher is compact and specialized in relevance scoring, making it far cheaper to supervise at scale than a general-purpose LLM.

We train our model with a weighted sum of a **document-level contrastive loss** and a **chunk-level distillation loss** over batches of query-document pairs. For each query, the

paired document serves as the positive, while the remaining documents in the same batch serve as negatives.

For each batch, we randomly select a chunking strategy, split the documents accordingly, and insert separator tokens between chunks. We then process each document in a single forward pass and mean-pool the token representations within each chunk to obtain contextual chunk embeddings. Queries are encoded separately, with mean pooling over their token representations. Finally, we compute the cosine similarity between every query embedding and every chunk embedding in the batch.

Loss function

Document-level contrastive loss

While our embedding model only produces chunk-level similarities, we can naturally derive document-level similarities from them. We consider a document d to be as relevant to a query q as its most relevant chunk. This scoring rule is inspired by the MaxSim operation of ColBERT, applied here to chunks instead of tokens.

Specifically, given a query q with embedding $\mathbf{q}$ and a document d as an ordered list of chunks c with embeddings $\mathbf{c}$, we define the document-level similarity as:

$$s(q, d) := \max_{c \in d} \mathbf{q}^\top \mathbf{c}.$$

Given these document-level similarities, we can formulate a document-level InfoNCE objective over in-batch negatives.

For a query q , let d^+ denote its relevant document and $\mathcal{D}$ the documents in the batch. The contrastive loss is as follows, omitting temperatures for readability:

$$\mathcal{L}_{\text{doc}}(q, d^+) = -\log \frac{\exp(s(q, d^+))}{\sum_{d \in \mathcal{D}} \exp(s(q, d))}$$

Chunk-level distillation loss

The contrastive loss teaches the model which document is relevant to a query. The distillation loss teaches it where relevance lies within that document, which is essential for a contextual model.

During training, we pass each positive query–document pair to our context compression model, which assigns a relevance score to every token in the document. As with document-level scoring, we derive each chunk’s relevance from its most relevant contents. To reduce sensitivity to outlier tokens, including those at chunk boundaries, we define chunk-level relevance as the mean of the top n token scores within the chunk.

For each query, we apply a temperature-scaled softmax to the teacher’s chunk-level relevance scores within the positive document to obtain a target distribution. Chunks in the other documents in the batch receive zero target probability. We train the embedding model to match this target by minimizing the forward KL divergence between the target and its predicted distribution over all chunks in the batch:

$$\mathcal{L}_{\text{dist}}(q, d^+) = \text{KL}(p^T \parallel p^S)$$

$$p^T(c_j) = \frac{\exp(r_j)}{\sum_{c_k \in d^+} \exp(r_k)}$$

$$p^S(c) = \frac{\exp(\mathbf{q}^\top \mathbf{c})}{\sum_{d \in \mathcal{D}} \sum_{c_k \in d} \exp(\mathbf{q}^\top \mathbf{c}_k)}$$

Here, the superscripts T and S indicate the teacher and student, and r_j is the teacher’s relevance score for chunk c_j in the positive document d^+ . The teacher’s target distribution assigns zero probability to chunks in all other documents in the batch.

Unlike a single gold-chunk label, this target captures degrees of relevance. Supporting chunks receive weight according to the teacher’s scores rather than being treated as negatives, teaching the model to retrieve supporting context as well as the answer.

Combined loss

Our combined training objective is a weighted sum of the document-level contrastive loss and the chunk-level distillation loss. We average this weighted loss over the batch

of query–document pairs.

Figure 2 illustrates our training approach. The context compressor serves as a teacher only during training. At inference time, the embedding model produces one contextual embedding per chunk, with no additional compression or reranking stage. The approach improves how contextual embeddings are learned without increasing index storage or adding teacher-model latency at retrieval time.



Figure 2: The training setup of our model. The contextual encoder (student, left) processes documents, each consisting of several chunks. Each chunk is embedded with respect to its surrounding document context. Query A is a known match for Document A and is scored against chunks of all documents to obtain predicted similarities. These chunk-level scores are max-pooled to document-level scores. We train chunk-level predictions and document level predictions to match target distributions (right). The chunk-level target is computed at training time as the prediction of the context compressor within the known relevant document and is set to zero within the known irrelevant documents. The document-level target distribution is set to be one-hot on the known relevant document.

Training data

The training data consists of roughly 430 public and in-house query–document datasets covering over 50 languages.

We construct in-house pair datasets from PII-filtered production data and by synthesizing relevant queries over long documents. For each forward pass, we select one dataset and sample an entire batch from it to increase the difficulty of in-batch negatives and avoid shortcut learning. None of our training datasets carry chunk-level annotations: all within-document supervision comes from the compression model, and no ConTEB data is used for training. Moreover, no context-bench data was available during the model development.

Training setup

The model starts from an in-house, 9B-parameter ColBERT retrieval model and produces 2048-dimensional embeddings through a linear projection layer. We mark chunk boundaries with a learned `<|chunk_sep|>` token and form each chunk embedding by averaging its token embeddings. Queries are encoded by the same model, with their token embeddings averaged into a single vector.

We use Matryoshka training to support both 1024- and 2048-dimensional embeddings, and quantization-aware training to support native int8 embeddings. During training, we gradually increase the learning rate, hold it constant, then reduce it.

The released model is a model soup that averages the weights of several checkpoints from the same training run.

A new benchmark for chunk-level retrieval

Public contextual-retrieval benchmarks such as ConTEB share the assumption behind gold-chunk supervision: each query has a single relevant chunk and a model is rewarded only for ranking that chunk first. They also inherit its limitations.

First, ConTEB tests retrieval on tasks where a chunk is ambiguous without its surrounding context. On these tasks, contextual models outperformed models that encode each chunk independently. However, it does not ask whether a model also surfaces the other parts of the document that make the chunk understandable.

Second, it does not systematically test whether the right document can be found when near-identical documents in the same corpus differ only in the context that surrounds

an otherwise identical sentence.

Both capabilities matter when a search agent or user must understand and verify an answer from a few retrieved sentences rather than an entire document. We therefore built context-bench, a controlled benchmark for retrieval over long documents, designed to distinguish the use of document context from general retrieval quality. It tests whether contextual embeddings use information from elsewhere in a document to support three capabilities:

- 1. **Document disambiguation:** Identify the correct document among near-identical alternatives.
- 2. **Answer retrieval:** Retrieve answer-bearing chunks whose meaning depends on details elsewhere in the document.
- 3. **Evidence retrieval:** Retrieve supporting chunks that clarify or substantiate the answer.

Context-bench contains 2,099 queries and 38,894 documents. Sentence length chunks are used, resulting in 2,458,072 total chunks. The queries cover 21 domains including corporate filings, clinical research, travel, entertainment, education, government, legal contracts, and software documentation.

Each query is designed to test one of twelve contextual capabilities. The primary target documents are intentionally long: 1,061 of the 1,197 distinct primary target documents have at least 200 sentences, with a median length of roughly 6,100 tokens. Every query has exactly one correct answer in the corpus. Where the same fact is stated more than once in a gold document, for example in a summary sentence and a table row, any of those sentences are accepted as answers.

An answer route is one valid combination of an answer chunk and the supporting evidence needed to interpret or verify it. Across the 2,099 queries, the primary target-document answer route has one evidence group for 964 queries, two for 735, three or more for 332, and none for 68. Alternative valid answer routes can require different evidence groups.

Table 1: Benchmark statistics.

Statistic	Value
Queries	2,099
Documents	38,894
Sentence chunks	2,458,072
Median primary target document length	6,100 tokens
Domains	21
Evidence groups per query (primary target-document answer route)	1: 964; 2: 735; 3+: 332; none: 68

Table 2: Twelve contextual capabilities tested by context-bench, with illustrative examples.

Capability	Example query	Answer chunk	Required document context
1. Entity identity	How much did Northlake spend on share repurchases?	The company repurchased \$12.8 billion of shares.	The introduction identifies Northlake as “the company.” A similar Southlake report is not relevant.
2. Period or version	Which cartridge fits the M40 revision B?	Install the KC-42 cartridge.	The applicability section says these instructions cover revision B, not revision C.
3. Definitions and aliases	What withdrawal rate did the 15 mg group have?	Zeta had a withdrawal rate of 12.7%.	The report defines Zeta as the 15 mg group. Without that definition, the chunk does not identify the requested group.
4. Table structure	What is the Max model’s battery capacity?	Typical capacity: 4,250 4,700 5,150.	The table header establishes Mini Base Max, measured in mAh. The answer is therefore 5,150 mAh.
5. Conditions and exceptions	How long is battery coverage for commercial use?	Battery coverage lasts 18 months.	A section introduction restricts this coverage to commercial installations;

			household installations have a different warranty.
6. References and pronouns	When did Mara become deputy mayor?	She took office in 2021.	The preceding narrative establishes that “she” refers to Mara, not Elin, who is also discussed.
7. List order and position	Which song opened the encore?	14. Lantern Road.	A note elsewhere says songs 14–16 constituted the encore. Track 14 is therefore its opening song.
8. Section scope	Which lens was used for the film’s cinematography?	A 35 mm prime was used throughout.	This passage belongs to “Principal Photography,” not the separate section about publicity photographs.
9. Speaker attribution	What reopening date did the mayor promise?	We will reopen on June 12.	The transcript identifies the mayor as the speaker. The same words from a contractor would not establish the mayor’s promise.
10. Changes over time	What notice period applied after the May amendment?	The notice period is now 45 days.	The amendment history establishes that this change took effect in May, replacing the earlier 30-day rule.
11. Causal relationships	Why did the west pump stop?	That fault caused the shutdown.	The investigation identifies “that fault” as a seized bearing in the west pump, and distinguishes it from an unrelated inlet obstruction.
12. Figurative or cross-language meaning	Figurative: What does “the winter room” symbolize in the poem? Cross-language: Which component does the French instruction tell the technician to remove?	Figurative: It represents exile, rather than a physical room. Cross-language: Retirez le capot. (Remove the cover.)	Figurative: The commentary identifies “it” as the poem’s recurring “winter room” image. Cross-language: The guide’s glossary specifies that “capot” means the protective motor hood, not the shipping cover.

Documents and queries

The queries, documents and capabilities being tested are inspired by conversations with turbopuffer's customers.

To give one example, consider the search needs of a property management firm. These firms have hundreds, sometimes thousands of lease agreements that differ only in the tenants names, dates, and a few other key details. A common query for that dataset might be “when does 5 park avenue’s lease end and what is the current rent?”. Lease agreements can also be lengthy and the critical disambiguating components that would confirm you’ve retrieved the right document (like the date, address, tenant name, etc.) can often be very far away from the key answer sentence containing the monthly lease payment amount. In practice, identical sentences like “Monthly rent is \$X,XXX.” can appear across many documents in the corpus.

A good contextual embedding model should use context from across the document to identify the correct document and retrieve both the answer and its supporting sentences. This lets a user or search agent quickly verify the answer and confirm that it comes from the right document.

Answer and evidence labels

For each query, we label “answer” and “evidence” sentences in the target document. A model passes our “reader sufficiency” test if it retrieves an answer sentence together with at least one sentence from each evidence group. Together, these sentences provide enough context for a reader to verify the answer with supporting evidence.

When contextual embedding models do this well, they can dramatically reduce the number of tokens search agents need to review as well as make it much easier for a person to quickly confirm that they have located the correct answer to their original query without having to read the entire document.

Every evidence group must be recovered, but any one of a group’s alternatives is enough to recover it. It’s common to see the same fact stated in several places across these lengthy documents. Retrieving just one sentence from each evidence group meets the reader sufficiency criteria and as such would be accepted as perfect evidence recall.

Metrics

Every document is chunked into sentences, and the same boundaries are used for all models. Two conditions are tested for non-contextual embedding models: one where each sentence is embedded independently and another where the entire document is embedded as a single chunk.

We use the sentence-level condition to measure answer recall and evidence recall. It tests whether a non-contextual model can identify the correct answer and supporting evidence when each sentence is embedded without access to the surrounding document. This is intentionally challenging. In many queries, the answer sentence alone lacks the details needed to distinguish it from similar sentences elsewhere in the corpus.

We use the document-level condition as a document recall baseline. It tests whether fine-grained sentence embeddings provide enough retrieval benefit to justify their additional storage cost relative to storing one vector per document. A sentence-level index creates substantially more vectors than a document-level index, so it should outperform the one-vector-per-document baseline to warrant that cost.

All evaluations use exhaustive rankings against all 2,458,072 chunks. Differences between models therefore reflect the representations, not index configuration. We focus on three key metrics:

Document@K is defined as:

$$\text{Document@K} = 1[\text{rank}(d^*) \leq K]$$

where d^* is the highest-ranked document in D_q .

Answer@K is defined as:

$$\text{Answer@K} = 1[A_q \cap C_K(q) \neq \emptyset]$$

where $C_K(q)$ is the top K chunks over the whole corpus and A_q is the set of accepted answer chunks.

Evidence Recall@K is measured within d^* as follows: Rank the sentences of d^* by similarity, remove the highest-ranked answer sentence, and let $R_K(q)$ be the top K that remain. For evidence groups $G_q = \{g_1, \dots, g_m\}$, group g_j is recovered if some alternative $E \in g_j$ satisfies $E \subseteq R_K(q)$, and

$$\text{EvidenceRecall@K} = 1[\text{rank}(d^*) \leq 10] \cdot \frac{1}{m} \sum_{j=1}^m 1[g_j \text{ recovered}].$$

Document@K asks whether contextualization lets the right document beat siblings and distractors that share its vocabulary.

Answer@K asks whether the answer sentence itself ranks among the top chunks.

Evidence Recall@K asks how much of the supporting context is recovered among the correct document’s top K chunks, after excluding the highest-ranked answer chunk. Removing the answer keeps evidence recall from double-counting what **Answer@K** already measures, and leaves all K slots for supporting context. Requiring the gold document to be in the top 10 retrieved documents means no model receives evidence credit for a document it would not have surfaced. We also report **All-Evidence@K**, the fraction of queries with every group recovered.

Together, these metrics decompose the challenges contextual-embedding models face: find the right document, find the right answer, and recover the minimal context that shows why the answer is correct.

Figure 3 illustrates the metrics with the property management example.

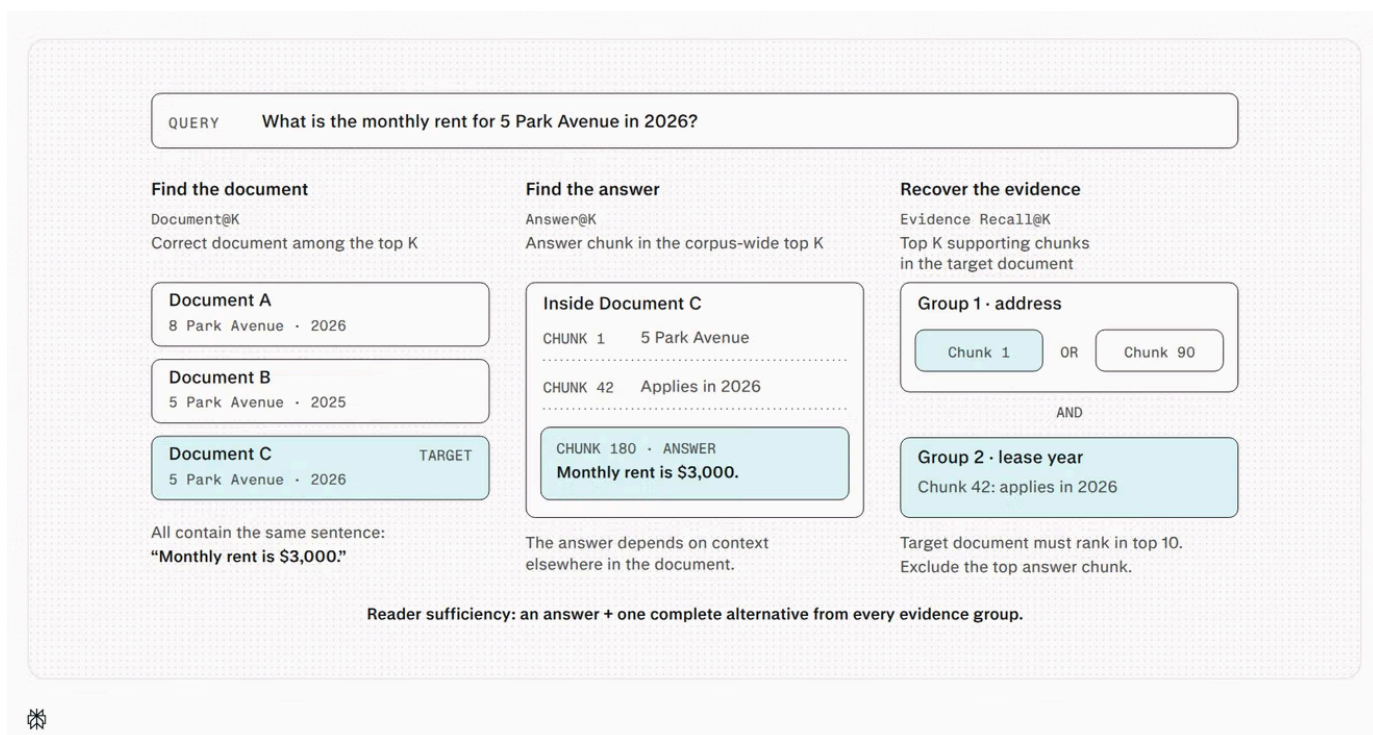


Figure 3: Context-bench metrics illustrated with a lease query. Document@K tests retrieval of the correct document; Answer@K tests retrieval of its answer chunk; Evidence Recall@K measures recovery of supporting evidence groups within the target document. In this example, verifying the rent requires both address and lease-year evidence. Evidence recall excludes the top answer chunk and requires the target document to rank in the top 10.

Evaluation results

We evaluate our model against contextual and non-contextual embedding models on four fronts.

Our contextual baselines are pplx-embed-context-v1-4B and voyage-context-4. Our non-contextual baselines are voyage-4-large and NVIDIA’s Nemotron-3-Embed-8B-BF16, which encode chunks independently. The baseline set varies by experiment, as indicated in each figure.

We start with context-bench, which directly tests the capabilities that motivate this work, and then report results on ConTEB, the standard public benchmark for contextual embeddings. To check that these gains do not come at the expense of general retrieval quality, we also evaluate domain-specific document and chunk retrieval benchmarks that we built from public datasets. Finally, we examine two practical properties for deployment: retrieval quality versus storage costs per vector, and sensitivity to the chunk size used at indexing time. All evaluations encode documents of up to 32,768 tokens in a single pass.

Context-bench

As shown in [Figure 1 in the introduction](#), our preview leads on Answer@K and Evidence Recall@K at every reported cutoff, as well as All-Evidence@10. At K = 10, it achieves 45.5% answer recall, 40.6% evidence recall, and 31.1% all-evidence recall. It also leads on Document@1 and Document@10, reaching 15.2% and 61.6%, respectively. At K = 10, our preview exceeds voyage-context-4 by 14.4 percentage points in answer recall and 5.0 points in evidence recall.

ConTEB

Our preview model achieves the highest average nDCG@10 among the models shown on ConTEB in Figure 4. It does not lead every task: pplx-context-v1-4B scores higher on NarrativeQA, while Nemotron-3-8B scores highest on COVID-QA. As the main authors also pointed out in the ConTEB paper, COVID-QA includes mostly query-chunk pairings that are very extractive and can be satisfied with simple surface lexical matching on technical medical terms. As a result, context understanding is less useful for this task and non-contextual models typically outperform contextual ones.

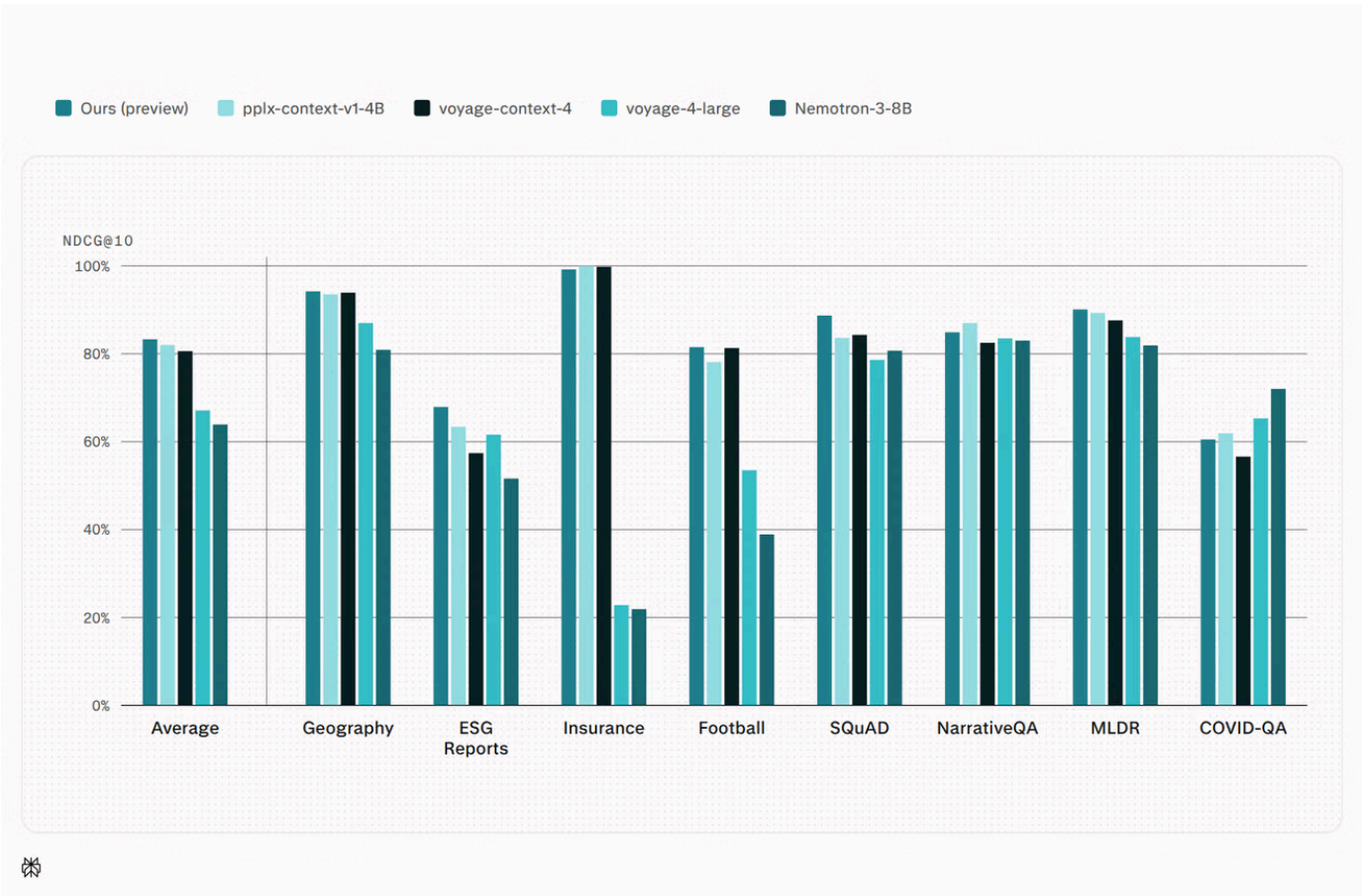


Figure 4: Task-level nDCG@10 on ConTEB in the query-to-chunk setting. The average is the unweighted mean across the eight tasks.

General retrieval quality across domains

Beyond benchmarks designed for contextual embeddings, we assess the domain-specific performance of our model.

Our query-to-chunk (Q2C) suite uses MTEB tasks such as LegalBench and FinanceBench alongside additional retrieval tasks we curated from public datasets. Within each relevant document, a chunk is labeled relevant if it contains the exact answer span when the task provides one, and otherwise by an LLM judging its relevance to the query. The query-to-document (Q2D) suite uses the same tasks and scores whether the relevant document is retrieved.

Our model achieves the highest average score on chunk retrieval in Figure 5 and leads in the legal, technical and health domains. It does not lead in every domain: voyage-context-4 scores higher in finance and multilingual retrieval, and Nemotron-3-8B scores highest in conversation retrieval. On document retrieval in Figure 6, our model scores slightly below voyage-context-4 on average, while leading in the technical domain. The pplx-context-v1-4B baseline scores lower on average in both suites, which is consistent with it having been trained only on the ConTEB training set, showing that supervision tied to one benchmark's gold chunks transfers poorly to other domains.

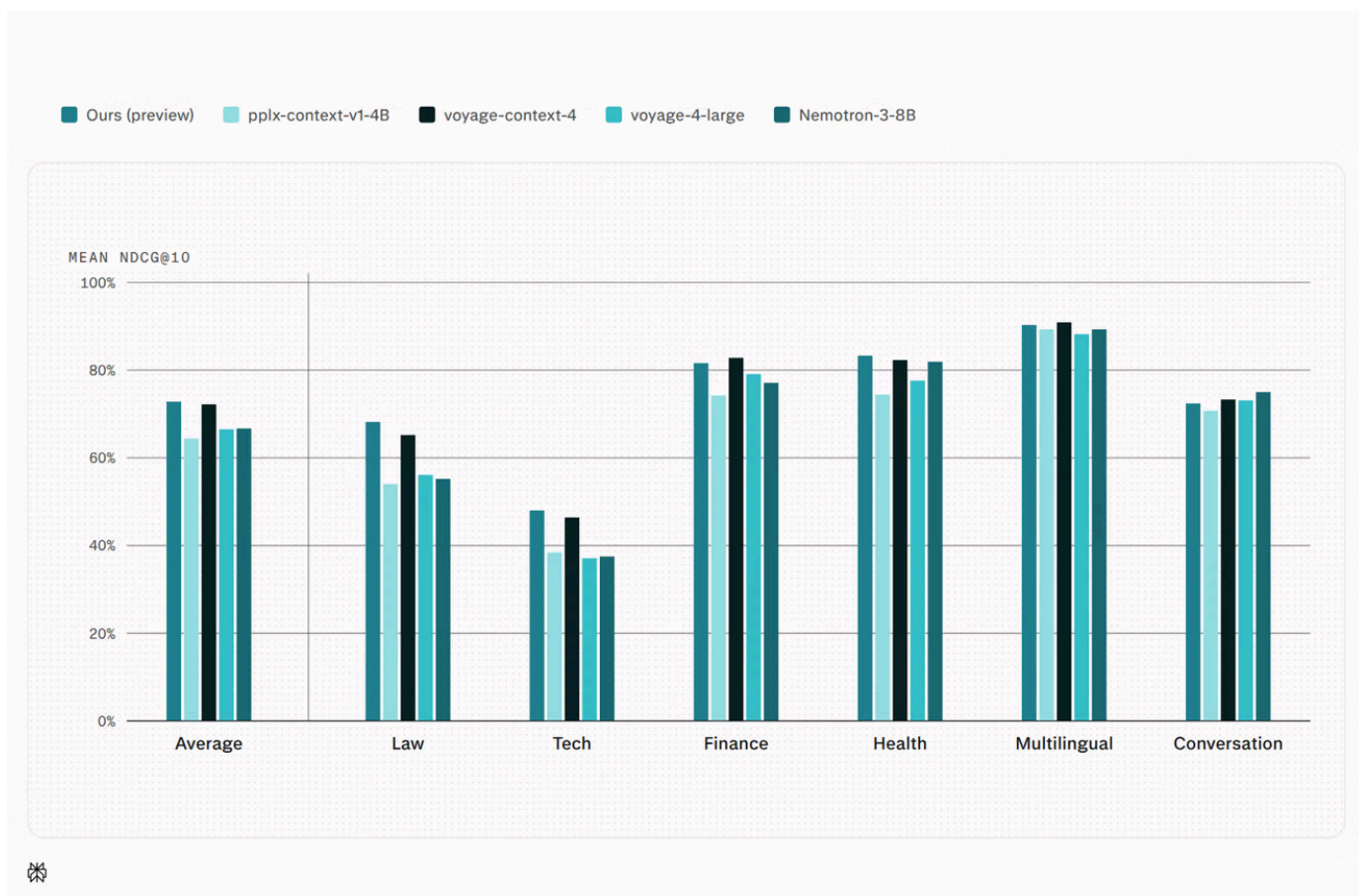


Figure 5: Mean nDCG@10 on the query-to-chunk (Q2C) tasks, grouped by domain. Domain bars are unweighted means of the task-level scores in that domain; the overall average is an unweighted mean across all displayed tasks.

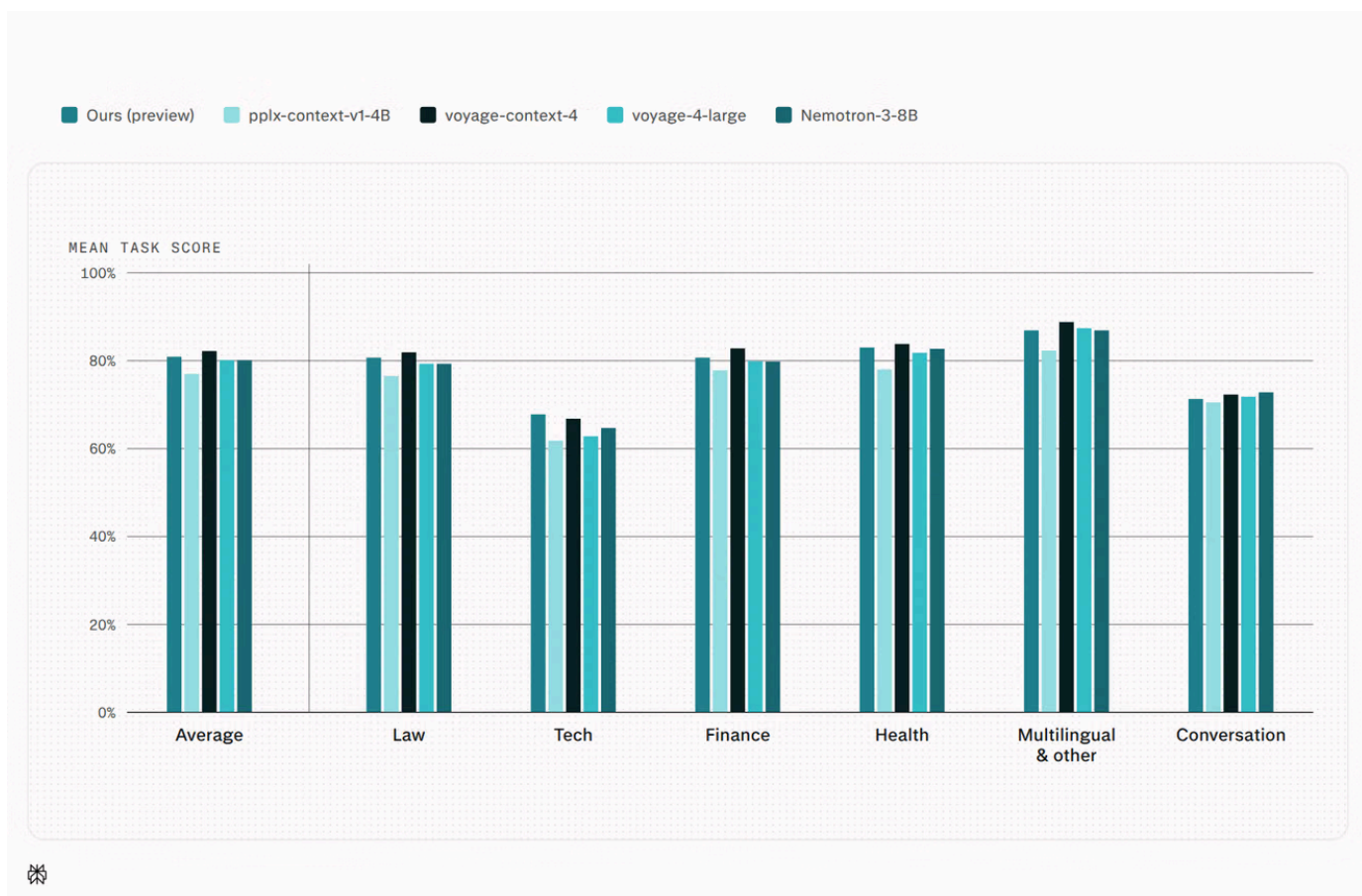


Figure 6: Mean retrieval score on the query-to-document (Q2D) tasks, grouped by domain. Each bar in a domain is the unweighted mean of its task-level main scores; the overall average is an unweighted mean across all displayed tasks.

Retrieval quality and storage costs

A contextual embedding model produces one vector per chunk, so an index built with our model contains exactly as many vectors as a conventional chunk-level index with the same chunking strategy. Contextualization changes what each vector encodes, not how many vectors are stored. The storage footprint is therefore governed by the size of each vector. The model offers two ways to reduce it: truncating the embedding from 2048 to 1024 dimensions and quantizing it to int8.

Figure 7 shows a favorable tradeoff between retrieval quality and vector storage. At 1024 dimensions, our int8 embeddings use 1 KB per vector while slightly exceeding the chunk-retrieval score of voyage-context-4 at 2048 dimensions in float32, which uses 8 KB per vector. Increasing our embedding size to 2048 dimensions raises retrieval quality further at 2 KB per vector. These storage figures refer to the vectors alone, not the rest of the index.

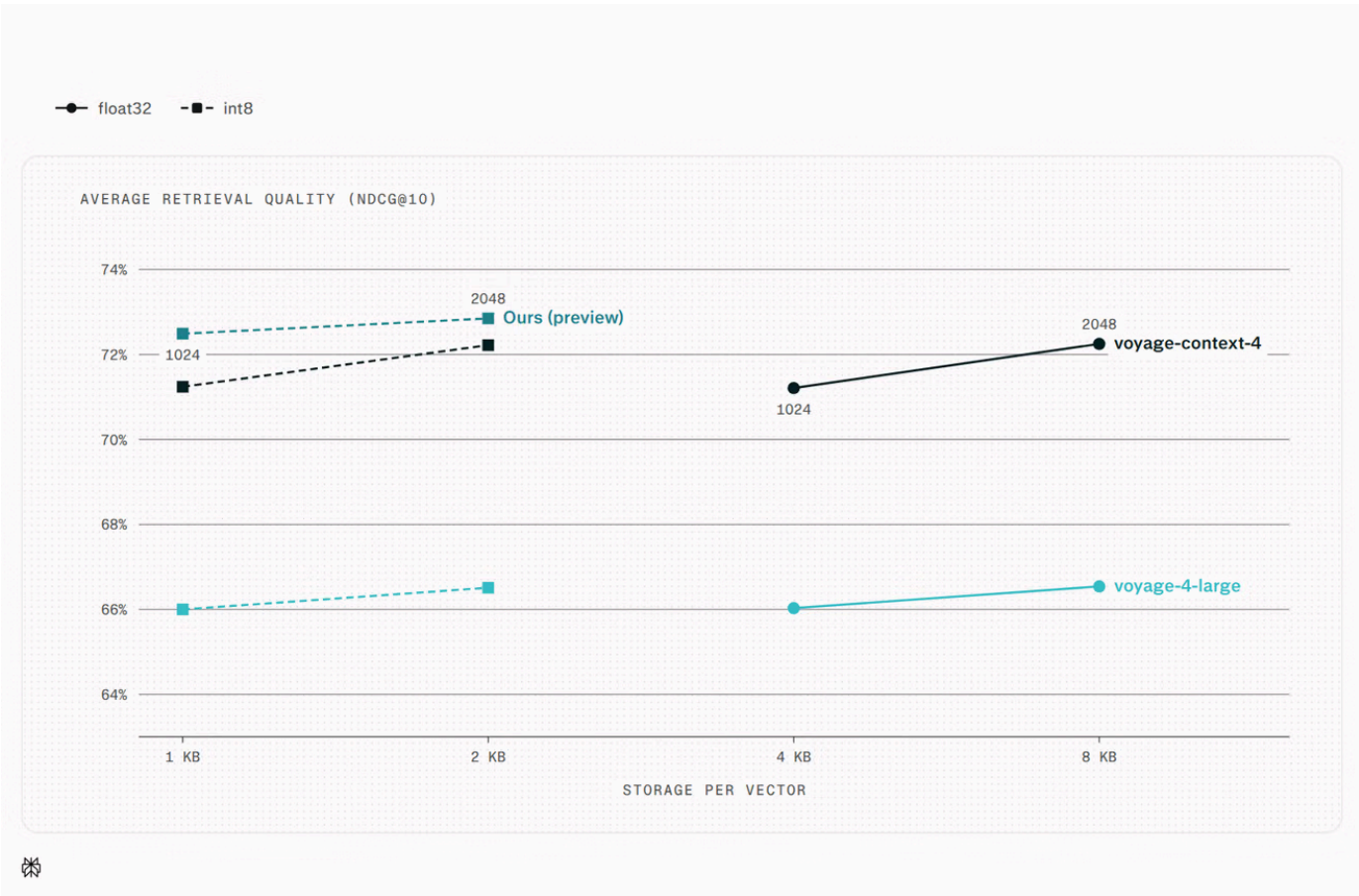


Figure 7: Chunk retrieval quality (average nDCG@10) as a function of storage per vector. Solid lines denote float32 and dashed lines denote int8; labels give the embedding dimension.

Chunk size sensitivity

Figure 8 shows only a modest change in retrieval quality as chunk size increases from 64 to 512 tokens: the average score falls from 81.0% to 79.9%, a decrease of 1.1 percentage points. This suggests that the model remains effective across the tested chunk sizes, giving indexing pipelines flexibility in how documents are split.

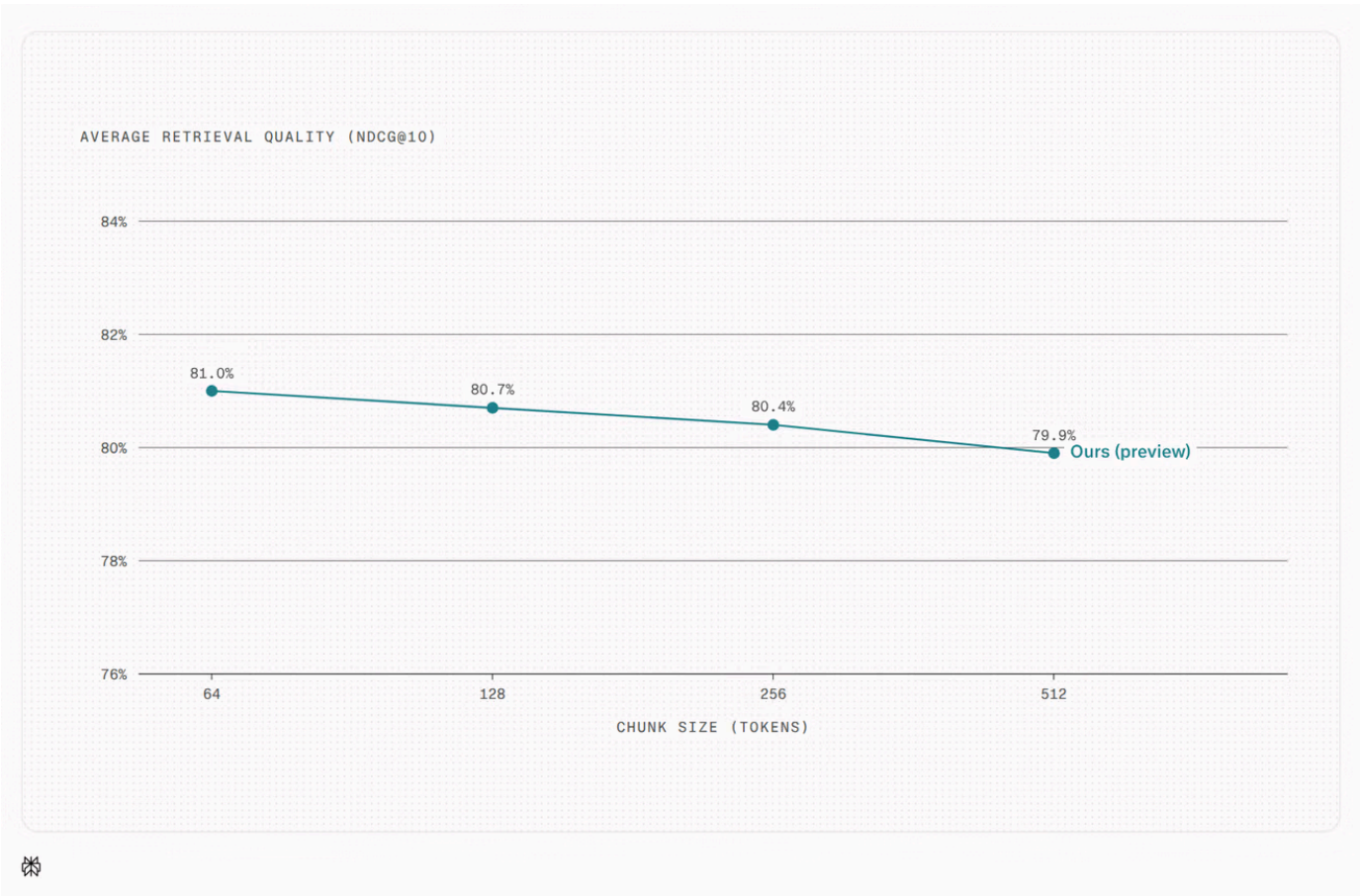


Figure 8: Chunk-size sensitivity of our model. Each point is the unweighted mean nDCG@10 across 74 MTEB tasks, with documents indexed at the given chunk size.

Release

turbopuffer keeps the full corpus, queries and labels of context-bench private and reports results for submitted models. A public benchmark of this kind invites direct training on its queries and documents, which would gradually saturate it and erode its value for tracking progress over time. If you would like your model to be evaluated on the benchmark, please contact turbopuffer at contextbench@turbopuffer.com.

A preview of our model is available on [Hugging Face](#), and all results in this post are reported for this preview. We are working on making the model available through the Perplexity API, and on further improving retrieval over long documents together with turbopuffer.

 Поделиться этой публикацией



ПРОДУКТЫ

Поиск

Computer

Comet

API

Глубокое исследование

РЕШЕНИЯ

ПО РАЗМЕРУ

Enterprise

ПО ОТРАСЛЯМ

Финансы

Юриспруденция

Консалтинг

Здравоохранение

Технологии

Государственные органы

Образование

Реклама

ПО КОМАНДАМ

Маркетинг

Продажи

Продукт

ИТ

ЦЕНЫ

РЕСУРСЫ

Pro

Блог

Max

Руководства

Enterprise

Клиенты

API

Академия

Мастер-классы

Документация API

Центр поддержки

Блог исследований

Безопасность

Центр доверия

КОМПАНИЯ

Вакансии

Руководство по фирменному стилю

Фирменный магазин

Политика конфиденциальности

Условия и положения



Perplexity ©2026

ИИ для
любознательных